Region Dynamic Models and External-Force Estimation for MABEL:

Gravity/Bias Feedforward, the Generalized-Momentum Observer, and Contact Localization from Joint Sensing (`mabel.dynamic_models`)

MABEL Project — Control & Teleoperation

July 24, 2026

Abstract

The `dynamic_models` subpackage answers, for each of MABEL's three control regions (*arms*, *upper body*, *whole body*), the two questions every torque controller asks: *what feedforward holds this region against gravity*, and *what external force or torque is someone applying to it*. One MuJoCo-backed oracle computes $g(q)$, the bias $C(q,\dot{q})\dot{q} + g(q)$, the mass matrix $M(q)$ and full inverse dynamics — region selection is pure joint indexing on top, so the physics can never diverge between regions. External interaction is estimated with a generalized-momentum disturbance observer (no joint accelerations needed), corrected for the constraint forces the observer cannot see but the model knows (joint friction loss, joint-limit reactions), and localized by hypothesis scoring over the arm links to a point ON the robot's surface with a pure point-force refit — arms only, since other regions would need system identification. Everything uses only $(q, \dot{q}, \tau_{\text{cmd}})$ and the model — all available on the real robot — so the identical pipeline runs in simulation and on hardware. Measured: gravity-comp holds the arms at 0.000 rad drift over 2 s where the uncompensated arm sags 1.01 rad; against ground-truth surface-point pushes (6 N to 20 N, both palms, forearm, elbow), palm forces are recovered to 0.5 N magnitude error at direction cosine 0.993 with the contact point 1.7 cm from truth on the correct link, and rest / post-push / moving-arm negative controls stay contact-free.

Keywords: inverse dynamics, gravity compensation, momentum observer, external wrench estimation, contact localization, mobile manipulation.

1 Overview

Rigid-body dynamics of the robot:

$$M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + g(q) \;=\; \tau_{\text{cmd}} + \tau_{\text{ext}}, \tag{1}$$

with joint torque command τ_{cmd} and external (operator) torque τ_{ext}. The subpackage is organised as:

- `model.py` — `MujocoDynamics`: $b(q,\dot{q}) = C\dot{q} + g$ from `qfrc_bias`; $g(q)$ from RNE at $\dot{q} = 0$; full inverse dynamics $\tau = M\ddot{q} + C\dot{q} + g$ (`mj_rne`); dense $M(q)$ (`mj_fullM`). Link masses and CoMs are runtime-adaptable (`set_link_mass` / `set_link_com` / `scale_total_mass`) so the model can track the real robot's uncertain inertial parameters.
- `observers.py` — the momentum observer (§4).

- `regions.py` + `arms/`, `upper_body/`, `whole_body/` — the region classes (§2).

The *plant* (state reads, effort writes, servo neutralisation) is deliberately outside this package (`mabel/plant.py`): the same dynamics drive `SimPlant` (MuJoCo `qfrc_applied`) and a future `RealPlant` (DAMIAO CAN torque commands, torque from motor-current sensing).

2 Control regions

region	torque-managed joints	lift	base
`arms`	16 (7 arm + ORCA wrist, both sides)	servo holds	holds
`upper_body`	17 (+ Torso hinge)	freed, gravity-comp'd	holds
`whole_body`	17 (same joints)	freed, gravity-comp'd	admittance

The ORCA hands' masses are part of the model, so arm gravity/bias terms already carry the hands; the finger joints themselves stay position-controlled (hand control is decoupled by design, `hand_retargeting.pdf`). The cascaded lift is tendon-driven: commands go to the `Lift` actuator while state and feedforward are read on its cascade joint (`config.LIFT_JOINT`); the region classes handle this mapping. The base contributes no torque joints — it is force-controlled through the admittance (`whole_body_controller.pdf`); the whole-body region therefore adds `base_wrench()` (§5) rather than more joints. A `RegionDynamicModel` exposes `gravity_ff()`, `bias_ff()` (as $\{\text{joint} \to \tau\}$), the region block of $M(q)$ (verified symmetric positive-definite in the tests), and `external_torques()` (§4). The wire protocol's legacy spelling `"arm"` aliases `"arms"`.

3 Gravity / bias feedforward

The feedforward that renders a region weightless is the bias evaluated at the measured state:

$$\tau_j^{\text{ff}} \;=\; b_j(q,\dot{q}) \;=\; \big[C(q,\dot{q})\,\dot{q} + g(q)\big]_j \qquad (\text{or } g_j(q) \text{ with } \texttt{use_bias=False}), \tag{2}$$

applied per managed joint with a small viscous damping $-D_j\dot{q}_j$ for numerical dissipation (0.6 Nm s on arm joints; 12 Nm s "lazy" damping on torso and lift so the big DOFs move deliberately). Substituting Eq. (2) into Eq. (1) leaves $M\ddot{q} \approx \tau_{\text{ext}} - D\dot{q}$: each joint behaves as damped pure inertia — it floats, holds pose, and can be dragged by hand. This is the $K{=}0$ limit of the impedance law and the foundation of both the gravity-compensation controller and the compliance controller (`whole_body_controller.pdf`).

3.1 One layer for every control space

Holding the robot up is a property of the *robot*, not of which control space the operator has selected, so the compensation is a single component (`dynamic_models/gravity.py`, `UpperBodyGravityComp`) shared by both reference generators — the kinematic stack (`teleop_engine`: Drive and whole-body Teleop) and the torque engine (`WBCEngine`: Joint and Task). It owns the canonical actuated upper body

$$\mathcal{U} \;=\; \{\text{Lift} \times 2,\ \text{Torso},\ \text{Neck} \times 3,\ \text{Arm 1–7} \times 2,\ \text{ORCA wrist} \times 2\}, \qquad |\mathcal{U}| = 22, \tag{3}$$

excluding the floating base and the swerve wheels, whose weight is carried by the ground contacts. Sharing the object is what makes the compensated set, the gain and the refresh rate identical in

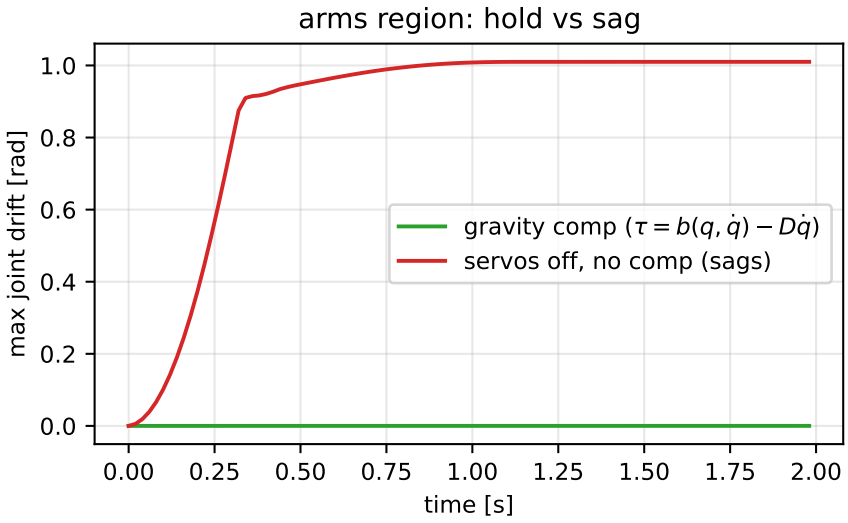


Figure 1: Arms region, servos neutralised: with the feedforward of Eq. (2) the arm holds (max drift 0.000 rad over 2 s); without it the arm sags 1.01 rad. Generated by `experiments/doc_module_figures.py`.

every mode *by construction* rather than by two implementations agreeing; the property is asserted directly in `tests/test_mode_switch_no_sag.py`.

Two implementation choices are load-bearing rather than incidental.

(i) Private scratch state. The torque is evaluated on the component's own `MjData`, never on the caller's. Computing $g(q)$ in place — zeroing and restoring `data.qvel` around `mj_rne` — is a mid-tick mutation of shared state that perturbs every consumer reading $\dot{q}$ in the same tick; measured at a 0.335 rad single-tick jump on `left_arm_5` during retarget re-acquire, tripping the step-free bound of `exp_real_retarget_chain.py`.

(ii) Evaluation at $\dot{q} = 0$, i.e. pure $g(q)$ with no Coriolis and no velocity feedback. This is what makes the layer safe on the ORCA wrists, which the torque law cannot hold. Their effective inertia is $\approx 2.5 \times 10^{-3}\,\mathrm{kg\,m^2}$, two orders below an arm link, so a $-D\dot{q}$ term computed from a state one RPC hop old lands out of phase and injects energy (§ `whole_body_controller.pdf`: the wrist rang to -118 rad/s and tunnelled through its MJCF limit). A static $g(q)$ has no velocity term to ring; its position sensitivity is bounded by $mgr \approx 1.74 \times 9.81 \times 0.05 \approx 0.9\,\mathrm{Nm/rad}$, orders below the servo gain it assists. Verified over 10 s of torque mode: peak wrist speed < 1 rad/s, both wrists inside their MJCF range.

The refresh is throttled to 30 Hz (one full-model `mj_forward` per recompute; several such callers oversubscribe the Jetson's cores), while the *write* happens every substep so no path can be left holding a stale vector. Gravity varies with pose, and at teleop joint speeds the pose moves negligibly in 33 ms.

4 The generalized-momentum observer

On hardware there is no acceleration signal and no force/torque sensor; the observer of De Luca & Mattone [1] recovers τ_{ext} from $(q, \dot{q}, \tau_{\mathrm{cmd}})$ alone. Define the generalized momentum $p = M(q)\,\dot{q}$.

Differentiating and substituting Eq. (1):

$$\dot{p} \;=\; \dot{M}\dot{q} + M\ddot{q} \;=\; \tau_{\text{cmd}} + \tau_{\text{ext}} - C\dot{q} - g + \dot{M}\dot{q}. \tag{4}$$

With the residual defined as

$$r(t) \;=\; K\Big(p(t) - p(0) - \int_0^t \big(\tau_{\text{cmd}} - b(q,\dot{q}) + \dot{M}\dot{q} + r\big)\,\mathrm{d}s\Big), \tag{5}$$

substituting Eq. (4) gives the residual dynamics

$$\dot{r} \;=\; K\left(\tau_{\text{ext}} - r\right) \quad\Longleftrightarrow\quad r \;=\; \frac{K}{s+K}\,\tau_{\text{ext}}, \tag{6}$$

a first-order low-pass estimate of the external joint torque with bandwidth K ($25\,\mathrm{s}^{-1}$ to $35\,\mathrm{s}^{-1}$ in use; higher = faster but noisier), requiring *no* $\ddot{q}$. Passive joint damping and springs are *model knowledge* (`qfrc_passive`, computable on hardware by `mj_forward` on the mirrored state), so they are counted as part of the modeled torque rather than left in the residual — otherwise every self-motion bleeds damping torque into the "external" estimate. Implementation notes: b comes from `qfrc_bias`; $\dot{M}\dot{q}$ from a finite difference of `mj_fullM` between ticks; the integral is explicit-Euler. `RegionDynamicModel.external_torques()` returns r indexed to the region's joints; τ_{cmd} must be the *full* commanded generalized torque — in simulation `qfrc_applied + qfrc_actuator`, on hardware the current-sensed motor torques.

5 Cartesian wrench and contact localization

(Consumed by the admittance; the implementation lives in the admittance subpackage as `wrench_estimation.py`.) A wrench $F \in \mathbb{R}^6$ applied at a body with Jacobian J loads the joints as $\tau_{\text{ext}} = J^\top F$. The regularized inverse

$$\hat{F} \;=\; \arg\min_F \; \|J^\top F - r\|^2 + \lambda^2\|F\|^2 \;=\; \left(JJ^\top + \lambda^2 I\right)^{-1} J\,r, \tag{7}$$

with $\lambda = 3 \times 10^{-2}$. *Where* the force acts — and how hard, honestly — is decided by *hypothesis scoring* over the arm's links (residual pattern matching in the spirit of Haddadin's collision identification [2]). For each candidate link k, distal → proximal:

1. a 6-DOF DLS wrench $(\hat{F}_0, \hat{M}_o)$ at the link origin o from the joints proximal to k reconstructs the force's *line of action*: every point p on it satisfies $\hat{M}_o = (p - o) \times \hat{F}_0$, with minimal-norm point

$$p_\ell \;=\; o + \frac{\hat{F}_0 \times \hat{M}_o}{\|\hat{F}_0\|^2}; \tag{8}$$

2. the line is *raycast into link k's own geometry* (mesh geoms via `mj_rayMesh`, primitives via `mju_rayGeom`), entering from outside along the push direction: the candidate contact point p_k is ON the robot's skin, not floating on the line (fallback: the line through the link origin, then the origin itself);

3. a *pure point force* is refit at p_k against the residual of the *whole arm*, $\hat{F}_k = (AA^\top + \mu I)^{-1} A\,r$ with $A = J_p(p_k)[:, \text{arm}]$ $(3 \times n)$, $\mu = 10^{-3}$. The pure-force refit is what keeps magnitudes honest: a 6-DOF wrench fit on few proximal joints soaks the residual into the couple and reports $\|\hat{F}\| \approx 0$ (measured on a forearm push: 0.1 N for a 10 N truth);

4. the hypothesis is scored by the unexplained residual $\|A^\top \hat{F}_k - r_{\text{arm}}\|$ — a wrong link cannot reproduce the loaded/unloaded joint pattern. Adjacent links can score within noise of each other, so the *incumbent* (currently tracked) link keeps the contact unless a challenger scores 1.25× better — hysteresis in hypothesis space, which stops the reported contact from strobing between neighbouring links.

Two *constraint-force attributions* precede localization, both pure model knowledge. **Friction loss**: MuJoCo (like the real DAMIAO joints) puts Coulomb friction in `qfrc_constraint`, invisible to the observer; it silently absorbs up to f_ℓ (0.5 Nm per arm joint) of any external torque. For a moving joint $\tau_{\text{ext}} = r + f_\ell \operatorname{sign}(\dot{q})$; for a static one $\tau_{\text{ext}} = r + f_\ell \operatorname{sign}(r)$, ramped in only for $|r| > f_\ell$ — after any arm motion, static friction freezes a standing, spatially smooth residual of up to f_ℓ *per joint* ($\|r\| \sim 2$ Nm arm-wide) that fits a fake ∼4 N contact exactly as coherently as a real push; the only separating feature is that per-joint it never exceeds f_ℓ. Compensating below that line inflates it into a phantom. **Joint limits**: a joint parked on a stop carries an arbitrarily large reaction (measured: a trajectory commanded 0.14 rad past a limit produced a 13.7 Nm residual = a fake 60 N contact); residual of the only-possible reaction sign on a joint within 0.03 rad of its limit is attributed to the limit and discarded. The tracked estimate is stabilized by a low-pass on the residual ($\tau = 60$ ms; the DLS amplifies per-joint noise near threshold), a 4-sample confirmation for contact attack and link switches, and a 30-sample hold on dropout. `WholeBodyDynamicModel.base_wrench()` runs observer → attribution → per-arm localization → reduction of all contact forces to the chassis planar frame (f_x, f_y, M_z) — the drive signal of the base admittance — and exposes `.contacts` = [{body, joint, point, force, surface}]. Estimation is deliberately **arms-only**: contacts on the chassis, lift or torso are not claimed, since on the real robot those demand a system identification the platform does not have, while the arms' dynamics come from the same MJCF that drives control.

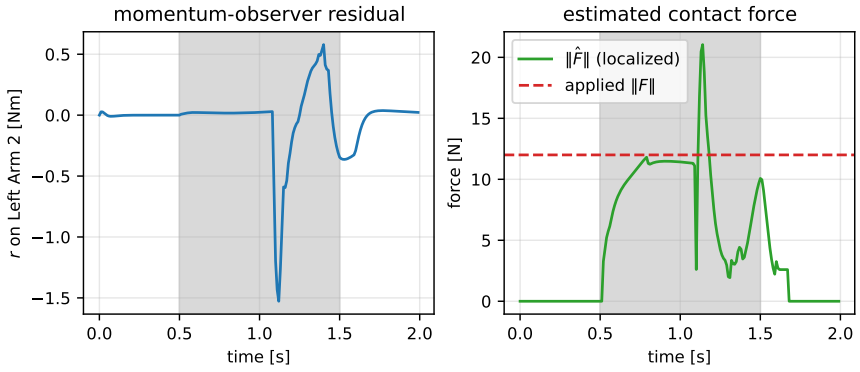


Figure 2: A 12 N world-x push on the left palm (grey window), gravity-comp'd upper body. Left: observer residual on `Left Arm 2` (Eq. (5)). Right: localized force-magnitude estimate (Eq. (7)–(8)) vs. the applied 12 N.

6 Experiments

From `experiments/doc_module_figures.py` and `tests/test_dynamic_models.py` (all headless MuJoCo on `scenes/mabel_home.xml`, settled 800 steps, $\Delta t = 2$ ms):

claim	condition	result
gravity-comp hold (arms)	Eq. (2), 2 s	max drift 0.000 rad
uncompensated sag (control)	servos off, 2 s	1.010 rad
lift hold (upper-body region)	freed lift, 1.5 s	drift <0.05 m (test)
push lands on correct arm	12 N on left palm	$\max_j \lvert r_j \rvert$: L > 3× R (test)
localized link/joint	same	`Left *` chain (test)
force direction (settled)	same	cos > 0.6 (test; measured 0.97)
force direction (mean over window)	same	cos = 0.42 (fig.; onset-diluted)
force magnitude	applied 12 N	$\hat{F}$ mean 9.3 N (fig.)
region $M(q)$ block	arms, 16 × 16	symmetric, $\lambda_{\min} > 0$ (test)

This scenario (a compliant, gravity-comp-only arm free-swinging under a sustained push at ∼1 rad/s) is the estimator's hardest regime: the window mean dilutes the onset transient and the 4-sample attack confirmation, and the filter chain (observer $1/K$, residual low-pass 30 ms, tracker blend) lags the rotating configuration. Settled values are cos ≈ 0.97 at 0.6–0.8× magnitude. The *stiff* (servo-held) regime is far better conditioned — the ground-truth sweep in `control_lab.pdf` measures 0.5 N magnitude error, cos = 0.993 and 1.7 cm contact-point error over ten surface-point pushes, with rest / post-push / moving-arm negative controls contact-free. The admittance consumer is robust to the compliant regime's bias: magnitude scales the glide speed (operator-in-the-loop), and the deadband absorbs the transient.

7 Implementation & usage

```
from mabel.dynamic_models import ArmsDynamicModel, WholeBodyDynamicModel
arms = ArmsDynamicModel(model, data)
ff = arms.gravity_ff()                          # {joint: Nm}, Day-2 PD feedforward
tau_ext = arms.external_torques(tau_cmd, dt)                  # momentum observer
wb = WholeBodyDynamicModel(model, data)
fx, fy, mz = wb.base_wrench(tau_cmd, dt); wb.contacts
```

On hardware, feed `MujocoDynamics` the *measured* $q, \dot{q}$ (kinematic mirror) and pass the current-sensed motor torques as τ_{cmd}; the class and equations are unchanged. Legacy imports (`mabel.dynamics`, `mabel.force_estimation`) resolve here through compatibility shims.

8 Limitations

- Estimation quality is bounded by model fidelity; the runtime mass/CoM adaptation hooks exist but no online identification is implemented.
- $\dot{M}\dot{q}$ uses a finite difference (noise at low Δt); the observer bandwidth K trades detection speed against that noise.
- Localization assumes at most *one* contact per arm and a point force (no torque about the contact); multi-contact returns the dominant one.
- The DLS wrench magnitude is biased low by λ (measured 0.74× at 12 N); direction is the trustworthy component.
- Hardware validation (DAMIAO current-sensed τ_{cmd}) is pending; all numbers here are simulation.

References

[1] A. De Luca and R. Mattone, "Sensorless robot collision detection and hybrid force/motion control," *Proc. IEEE ICRA*, 2005.

[2] S. Haddadin, A. De Luca and A. Albu-Schäffer, "Robot collisions: A survey on detection, isolation, and identification," *IEEE T-RO*, 33(6), 2017.

[3] R. Featherstone, *Rigid Body Dynamics Algorithms*, Springer, 2008.